NAME:S.DEVI SHREE

REGISTER NO:192511149

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems

COURSE CODE: CSA0503

COURSE OUTCOME COVERED

CO4: Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency. (BL3, BL4)

Activity Tool : Industry Problem Solving Task (Medium)

Assessment Tool Weightage: 35%

Code of Conduct:

I S.DEVI SHREE(192511149) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

1.An e-commerce company needs to store 10 million product records. Recommend a file organization strategy and justify your choice with cost analysis.

E-Commerce Company — 10 Million Product Records

Answer

For an e-commerce system with **10 million product records**, a **B+ Tree-based sorted/indexed file organization** is recommended.

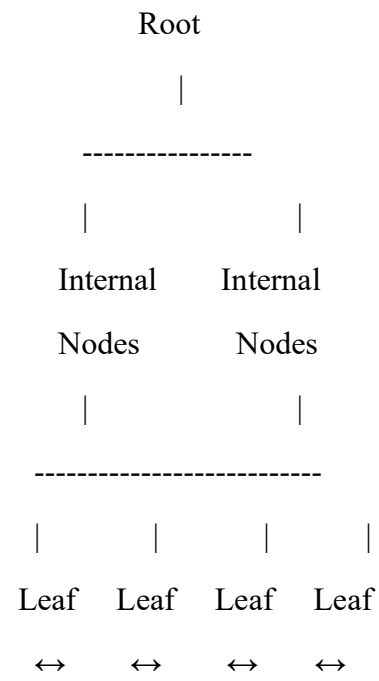
B+ Tree

An e-commerce database commonly requires:

- Search by ProductID
- Search by category
- Price-range queries
- Sorting by price
- Searching products within a price range
- Frequent product updates

A B+ Tree is suitable because it provides efficient search and range queries.

Structure



The leaf nodes are linked, making range queries efficient.

Cost Analysis

For N = 10,000,000 records:

B+ Tree search complexity:

where B is the number of entries per node.

If a B+ Tree has approximately 100 children per internal node:

Therefore, only about **4 tree levels** may need to be traversed.

Operation	**Approximate Cost**
Search	O(log N)
Insert	O(log N)
Delete	O(log N)

Operation	Approximate Cost
Range search	O(log N + k)
Sequential scan	O(N)

where k is the number of matching records.

Alternative: Heap File

Heap files have:

- Very fast insertion: O(1) approximately
- Poor search: O(N) without an index
- Poor range queries

For 10 million products, scanning the entire file for every search would be expensive.

Conclusion

B+ Tree indexed organization is recommended because it provides efficient search, insertion, deletion and range queries. Although it requires additional index storage and maintenance, the cost is justified for a 10-million-record e-commerce database.

2. A banking system requires fast retrieval of transactions by account number and date range. Design an indexing strategy with appropriate index types.

Banking System — Transactions by Account Number and Date Range

A **composite B+ Tree index** on (AccountNumber, TransactionDate) is the most suitable primary indexing strategy.

Proposed Index

(AccountNumber, TransactionDate)

Example:

Account No	Date	Transaction
1001	2026-01-05	T101
1001	2026-01-10	T102
1001	2026-02-15	T103
1002	2026-01-03	T104

Composite B+ Tree

A query such as:

SELECT *

FROM Transactions

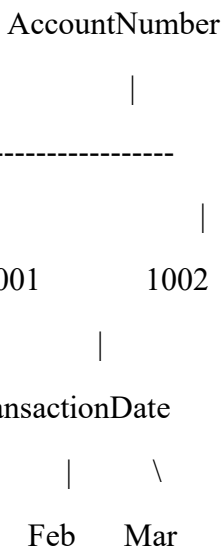
WHERE AccountNumber = 1001

AND TransactionDate BETWEEN '2026-01-01' AND '2026-01-31';

can efficiently use the composite index.

The index first finds account 1001 and then scans the date range.

Index Structure



Additional Index

If the system frequently searches by date alone, create another B+ Tree index:

(TransactionDate)

Cost

Operation	**Complexity**
Account search	O(log N)
Account + date range	O(log N + k)
Insert	O(log N)
Delete	O(log N)

k = number of transactions returned.

Conclusion

Use:

Primary composite index:

(AccountNumber, TransactionDate)

Optional secondary index:

(TransactionDate)

This provides fast account-specific and date-range transaction retrieval.

3.A healthcare system stores patient records accessed both by PatientID and by doctor name. Design a multi-level indexing solution.

Healthcare System — PatientID and Doctor Name

A **multi-level indexing solution using B+ Trees** can be used.

We need to support two different search fields:

1. PatientID
2. DoctorName

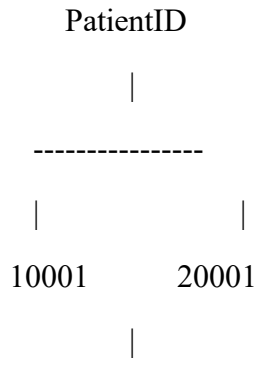
Therefore, create separate indexes.

Level 1 — Primary Index

Create a B+ Tree on:

PatientID

PatientID is usually unique.



Level 2 — Secondary Index

Create a secondary B+ Tree on:

DoctorName

Example:

+--- Dr. Kumar → Patient 1001, 1005, 1010

|

+--- Dr. Priya → Patient 1002, 1008

|

+--- Dr. Ravi → Patient 1003, 1007

Multi-Level Structure

Root Index

|

| |

PatientID DoctorName

B+ Tree B+ Tree

| |

Patient Data Patient Data

Search Performance

For N records:

For DoctorName, multiple matching patients can be retrieved efficiently using linked B+ Tree leaves.

Advantages

- Fast PatientID lookup
- Fast doctor-based lookup
- Efficient range queries
- Supports large databases
- Indexes can grow dynamically

Conclusion

Use **two B+ Tree indexes**:

Primary Index → PatientID

Secondary Index → DoctorName

This provides efficient access regardless of which attribute is used.

4. A logistics company needs to efficiently handle dynamic insertions and deletions. Analyze whether B-Tree or B+ Tree is more suitable.

Logistics Company — B-Tree vs B+ Tree

For a logistics system, **B+ Tree is more suitable than B-Tree**.

Comparison

Feature	**B-Tree**	**B+ Tree**
Data in internal nodes	Yes	No
Data in leaf nodes	Yes	Yes
Leaf nodes linked	Usually no	Yes
Sequential access	Good	Excellent
Range queries	Good	Excellent
Search	O(log N)	O(log N)
Insert	O(log N)	O(log N)
Delete	O(log N)	O(log N)
Fan-out	Lower	Higher
Database use	Less common	Very common

B+ Tree

Logistics systems frequently need queries such as:

Find shipment 50021

and:

Find all shipments from January 1 to January 10.

Point searches are efficient in both trees, but B+ Trees are better for range queries because the leaves are linked.

Dynamic Operations

When insertion causes overflow:

Full leaf

↓

Split

↓

Update parent

When deletion causes underflow:

Underflow

↓

Redistribute or merge

↓

Update parent

Conclusion

B+ Tree should be selected because it supports dynamic insertion and deletion efficiently while providing excellent sequential and range access through linked leaf nodes.

5. Design a hashing scheme for an HR system with 5000 employees. Evaluate static hashing vs. extendible hashing.

HR System — 5000 Employees

Hashing is suitable for employee lookup using a unique field such as:

EmployeeID

Use:

where M is the number of buckets.

Static Hashing

Suppose:

M = 1000 buckets

Hash function:

Example:

EmployeeID = 5025

5025 % 1000 = 25

Therefore, employee 5025 goes to bucket 25.

Advantages

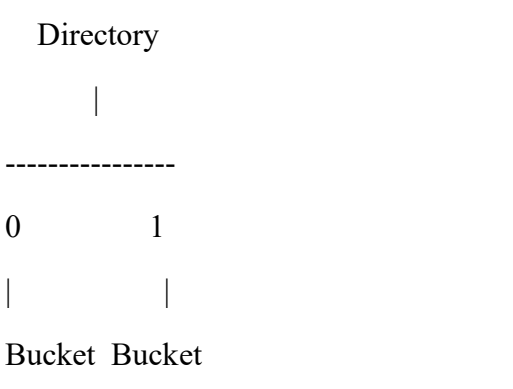
- Simple
- Fast average search
- Easy implementation
- Low overhead

Disadvantage

If employee records increase significantly, overflow buckets may be required.

Extendible Hashing

Extendible hashing dynamically increases the directory when buckets overflow.



Overflow

↓

Directory doubles

00 01 10 11

Comparison

Feature	Static Hashing	Extendible Hashing
Fixed size	Yes	No
Dynamic growth	Poor	Excellent
Overflow	Overflow chains	Bucket splitting
Search	O(1) average	O(1) average
Implementation	Simple	More complex
Suitable for growing HR DB	Moderate	Excellent

Recommendation

For exactly 5000 employees with predictable growth, static hashing can be used.

However, an HR system normally grows over time. Therefore **extendible hashing is the better long-term choice**.

6.An airline reservation system must support point queries by flight number and range queries by date. Propose a combined index structure.

Airline Reservation — Point and Range Queries

A combined indexing strategy should use **two B+ Tree indexes**.

Index 1 — Flight Number

Create:

B+ Tree(FlightNumber)

Used for point queries:

Find flight AI202

Complexity:

Index 2 — Date

Create:

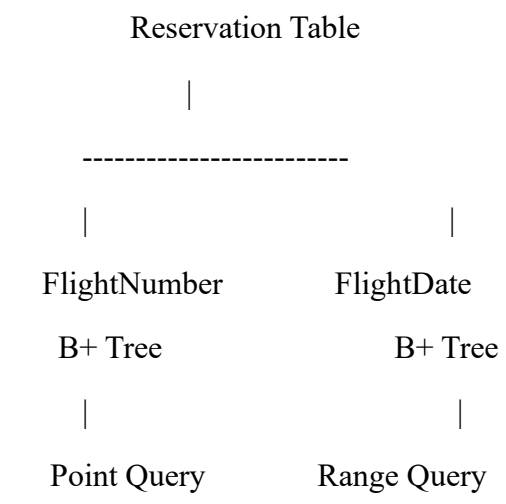
B+ Tree(FlightDate)

Used for range queries:

Flights between June 1 and June 15

Complexity:

Structure



Example

Flight search:

FlightNumber = AI202

uses:

Index(FlightNumber)

Date search:

FlightDate BETWEEN 2026-08-01 AND 2026-08-15

uses:

Index(FlightDate)

Conclusion

Two B+ Tree indexes provide the best solution:

B+ Tree → FlightNumber

B+ Tree → FlightDate

This handles both point and range queries efficiently.

7. For a university student database, design the file organization and secondary indexing to support fast queries by department and CGPA range.

University — Department and CGPA Range

Use a **heap or sorted file organization with secondary B+ Tree indexes**.

Since student records may be frequently inserted and updated, a **heap file + secondary indexes** is a practical choice.

File Organization

Student File

StudentID | Name | Department | CGPA

---

101 | Anu | CSE | 8.7

102 | Ravi | ECE | 7.9

103 | Priya| CSE | 9.1

Secondary Index 1 — Department

Create:

B+ Tree(Department)

Example:

CSE → 101, 103, 107

ECE → 102, 108

MECH → 104, 105

Secondary Index 2 — CGPA

Create:

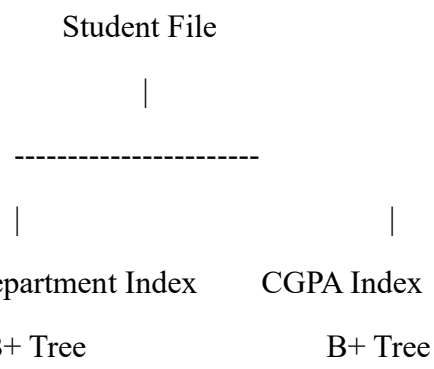
B+ Tree(CGPA)

For:

CGPA BETWEEN 8.0 AND 9.0

the B+ Tree can directly reach the first matching value and scan the linked leaves.

Structure



Complexity

Query	Complexity
Department search	O(log N + k)
CGPA range	O(log N + k)
Insert	O(log N)
Delete	O(log N)

Conclusion

Use:

Heap file + B+ Tree secondary index on Department + B+ Tree secondary index on CGPA.

This provides efficient department filtering and CGPA range searches.

8. Analyze disk block utilization for Heap, Sorted and Hashed file organizations given 100,000 records with 50 bytes each and 4KB blocks.

Disk Block Utilization — Heap, Sorted and Hashed

Given

Records = 100,000

Record size = 50 bytes

Block size = 4 KB = 4096 bytes

Step 1 — Records per Block

$$Records/block = \lfloor 4096/50 \rfloor$$
$$= 81$$

Therefore, each block stores **81 records**.

Step 2 — Number of Blocks

$$Blocks = \lceil 100000/81 \rceil$$
$$= 1235\ blocks$$

Step 3 — Utilization

Total record space:

100000×50=5,000,000 bytes

Total allocated block space:

1235×4096=5,058,560 bytes

Utilization:

≈ **98.84%**

Comparison

Organization	Blocks	Utilization	Search
Heap	1235	≈98.84%	O(N)
Sorted	1235	≈98.84%	O(log N)
Hashed	≈1235+overflow	Depends on load factor	O(1) average

Heap

Records are stored wherever space is available.

Advantage:

- Simple

- Fast insertion

Disadvantage:

- Slow search

Sorted

Records are maintained in sorted order.

Advantage:

- Binary search
- Efficient range queries

Disadvantage:

- Expensive insertion/deletion

Hashed

Records are distributed according to a hash function.

Advantage:

- Very fast equality search

Disadvantage:

- Collisions and overflow

Conclusion

All three have approximately the same basic data capacity when blocking is identical, but their **access performance differs significantly**.

9. A social media platform needs to index 500 million posts by user_id, timestamp and hashtag. Design a composite indexing strategy.

Social Media — 500 Million Posts

For **500 million records**, indexing must minimize disk I/O and support different query patterns.

The important attributes are:

- user_id
- timestamp
- hashtag

A single three-column index may not efficiently support every query. Therefore, use multiple carefully designed composite indexes.

Index 1 — User + Timestamp

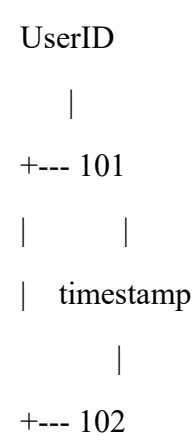
(user_id, timestamp)

This supports queries such as:

Find posts by user 105

between January 1 and January 30.

B+ Tree:



Index 2 — Hashtag + Timestamp

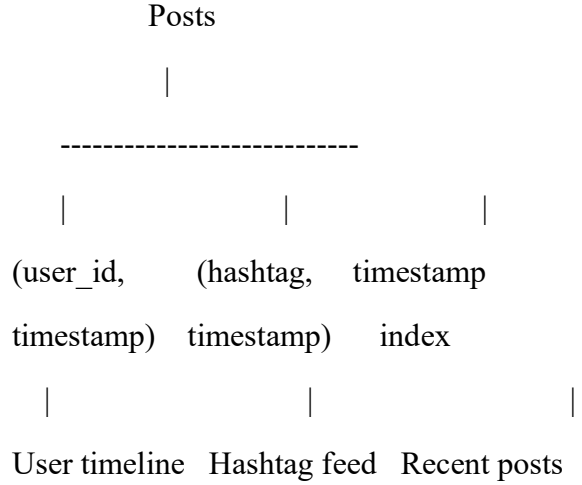
(hashtag, timestamp)

Index 3 — Timestamp

If the platform frequently retrieves globally recent posts:

can be maintained separately.

Proposed Design



Cost

For a B+ Tree:

where:

- N = 500 million posts
- k = number of matching posts

Important Consideration

With 500 million posts, indexes themselves consume considerable storage.

Therefore:

- Use compact index keys.
- Partition data by time.
- Consider sharding by user ID.
- Keep indexes only for frequently used query patterns.

Conclusion

Recommended indexes:

(user_id, timestamp)

(hashtag, timestamp)

(timestamp)

The first two are the most important because they support user timelines and hashtag-based time-range queries.

10. Compare the performance of Heap File, Sorted File and Hashed File for a supermarket billing system with 1 million daily transactions.

Supermarket Billing — Heap vs Sorted vs Hashed

Assume transactions have a unique:

TransactionID

The three organizations can be compared based on:

- Insert
- Delete
- Search

1. Heap File

Records are stored in no particular order.

Insert

O(1)

New transaction can be added at the end.

Search

O(N)

Requires sequential scan.

Delete

O(N)

First search for the record and then delete.

2. Sorted File

Records are maintained according to TransactionID.

Search

Binary search:

O(logN)

Insert

A record may have to be inserted at the correct position.

O(N)

Delete

Deletion may require shifting records.

O(N)

3. Hashed File

Use:

h(TransactionID)=TransactionID%M

Search

Average:

O(1)

Insert

Average:

O(1)

Delete

Average:

O(1)

assuming a good hash function and controlled load factor.

Comparison Table

Operation	**Heap**	**Sorted**	**Hashed**
Insert	**O(1)**	O(N)	**O(1) average**
Search	O(N)	O(log N)	**O(1) average**
Delete	O(N)	O(N)	**O(1) average**
Range query	Poor	**Excellent**	Poor
Storage overhead	Low	Low/Medium	Medium
Best use	Heavy insertion	Range queries	Exact lookup

For 1 Million Transactions

Suppose there are:

1,000,000 transactions

A heap search may require examining up to approximately:

1,000,000 records

in the worst case.

A sorted file requires approximately:

comparisons for binary search.

Hashing can locate the appropriate bucket directly, giving approximately **constant-time average access**.

Recommendation

For supermarket billing, if the most common operation is:

Find transaction using TransactionID

Hashed File is the best choice.

where **k = number of records returned**.